自 然 语 言 处 理

第5章 文本向量化（下）

授课教师：李芳芳
联系方式：lifangfang@csu.edu.cn

本章教学内容

文本向量化

本章内容一览表（共计4学时）

每小节概要	每小节知识点	重难点
第1小节 前景概要	为什么要进行文本向量化?	否
第2小节 离散表示：词袋模型	四种词袋模型：Onehot、TF、TF-IDF、Ngram	**是**
第3小节 分布式表示：词嵌入	为什么要进行词嵌入?	否
第4小节 Word2Vec	Word2Vec进行文本向量化的原理、优缺点	**是**
第5小节 ELMO模型	ELMO进行文本向量化的原理、优缺点	**是**
第6小节 GPT模型	GPT进行文本向量化的原理、优缺点	**是**
第7小节 课堂小结	内容小结	否

本章教学内容

文本向量化思维导图



4 词嵌入之Word2Vec

Word2Vec存在的问题?

- Word Embedding在对多义词进行编码的时候，是**区分不开多个含义的**，因为它们尽管上下文环境中出现的单词不同，但是在用语言模型训练的时候，不论什么上下文的句子经过word2vec，都是预测同一个单词；
- 而同一个单词占的是权重矩阵W中同一行的参数空间，这导致两种不同的上下文信息都会**编码到相同的word embedding空间**里去。
- 所以word embedding无法区分多义词的不同语义，这就是它的一个比较严重的问题。

4 词嵌入之Word2Vec

Word2Vec存在的问题?

HOW TO SOLVE

- 有很多研究人员提出很多方法试图解决这个问题，但是从今天往回看，这些方法看上去都成本太高或者太繁琐了，有没有简洁优雅的解决方案呢?



5 ELMO

ELMO名称的由来

- ELMo：Embedding from Language Models
- 其实这个名字并没有反应它的本质思想，提出ELMO的论文题目：“Deep contextualized word representation”更能体现其精髓。

5 ELMO

ELMO名称的由来

- Word2Vec一个最大的缺点就是无法处理一词多义的问题，因为每个词只有一个静态的词嵌入向量来表示词义。

⇩

- 上下文相关的词嵌入(Contextualized Word Embedding):

句子中的词都有一个单独的词向量（尽管有些词可能长得一模一样）

5 ELMO

ELMO名称的由来

- 之前的Word Embedding问题是：其本质上是个**静态**的方式，所谓静态指的是训练好之后每个词的表达就固定住了，以后使用的时候，不论新句子上下文的词是什么，这个词的Word Embedding不会跟着上下文场景的变化而改变。
- 比如Bank这个词，它事先学好的Word Embedding的某种表达，在应用中来了个新句子，即使从上下文中（比如句子包含money等词）明显可以看出它代表的是“银行”，但是对应的Word Embedding内容也不会变。

5 ELMO

ELMO名称的由来

- NLP领域两个著名的模型名称都来自《芝麻街》这个动画



5 ELMO

ELMO的核心要点

阶段一

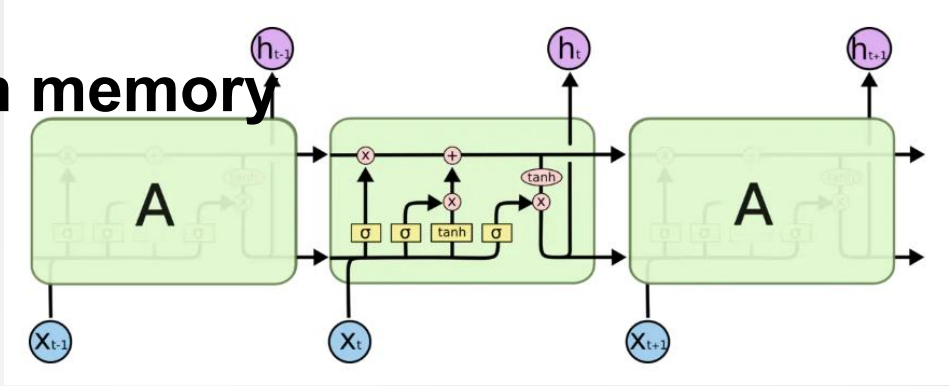
利用语言模型进行预训练：使用大规模的无标签语料库训练双向LSTM语言模型，ELMo（Embedding from Language Models）也是因此得名；

阶段二

将ELMo得到的特征向量送到下游任务中，得到任务相关的预测结果。

5 ELMO

LSTM：Long short-term memory



LSTM中重要的三个向量：
当前输入：X_t
当前输出：h_t
细胞状态：C_t

LSTM中重要的三个门：
(用于控制某个向量保留多少)
输入门：i_t
遗忘门：f_t
输出门：o_t

1、根据当前输入和上一时刻输出计算三个门：

$$i_t = \sigma(W_i \cdot [h_{t-1}, X_t] + b_i)$$
$$f_t = \sigma(W_f \cdot [h_{t-1}, X_t] + b_f)$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, X_t] + b_o)$$

2、计算当前细胞状态：

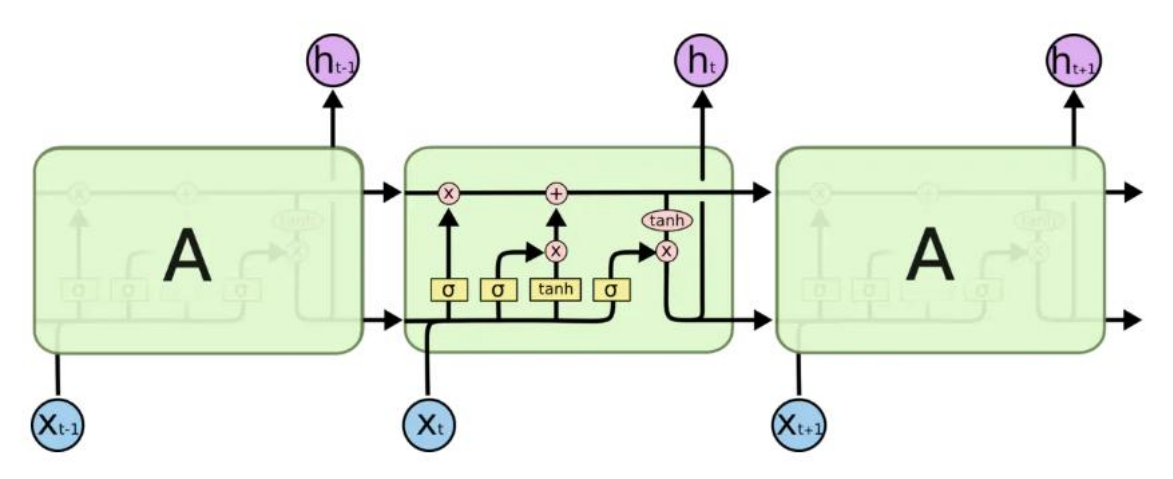
$$\widetilde{C_t} = \tanh(W_C \cdot [h_{t-1}, X_t] + b_C)$$
$$C_t = f_t \times C_{t-1} + i_t \times \widetilde{C_t}$$

3、计算输出值：

$$h_t = o_t * tanh(C_t)$$

5 ELMO

LSTM：Long short-term memory



简单的理解：每个LSTM单元的输入是一个词(使用One-Hot或Word2Vec等方式向量化)，输出是根据当前输入和以往有用的信息所确定的一个向量。可以看做把词向量进一步编码成了对当前任务更有用的向量。

5 ELMO

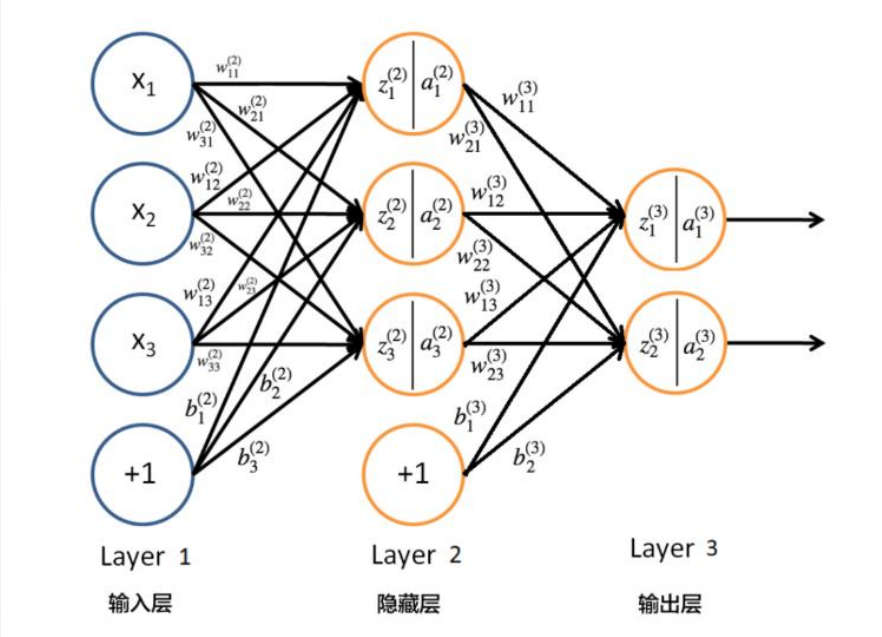
LSTM到RNN

要理解LSTM，首先需要理解RNN；因为LSTM是在RNN结构的基础上进行的改造！
RNN-based language model

RNN循环神经网络

RNN：Recurrent Neural Network，RNN

- 神经网络可以当做是能够拟合任意函数的黑盒子，只要训练数据足够，给定特定x，就能得到希望的y，结构图如下：



RNN循环神经网络

RNN：Recurrent Neural Network，RNN

- 将神经网络模型训练好之后，在输入层给定一个x，通过网络之后就能够在输出层得到特定的y，那么既然有了这么强大的神经网络模型，**为什么还需要RNN（循环神经网络）模型呢?**

RNN循环神经网络

- 他们都只能单独的去处理一个个的输入，前一个输入和后一个输入是完全没有关系的。但是，某些任务需要能够更好的处理序列信息，即前面的输入和后面的输入是有关系的。

> ➢ 比如，当我们在理解一句话的意思时，孤立的理解这句话的每个词是不够的，我们需要处理这些词连接起来的整个序列；
> ➢ 当我们处理视频的时候，我们也不能只单独的去分析每一帧，而要分析这些帧连接起来的整个序列。

RNN循环神经网络

- 以nlp的一个最简单的**词性标注任务**来说，将“我 吃 苹果”三个单词标注词性为 “我/nn 吃/v 苹果/nn”。
- 那么这个任务的输入就是：
  - 我 吃 苹果 （已经分词好的句子）
- 这个任务的输出是：
  - 我/nn 吃/v 苹果/nn(词性标注好的句子)

对于这个任务，我们当然可以直接用普通的神经网络来做，给网络的训练数据格式就是：
我-> 我/nn （多个单独的单词->词性标注好的单词）

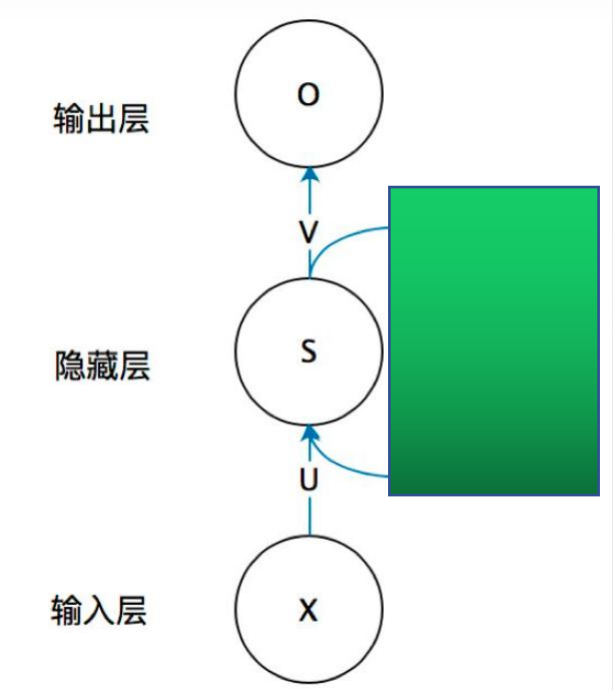
RNN循环神经网络

- 但是很明显，一个句子中，前一个单词其实对于当前单词的词性预测是有很大影响的，比如预测苹果的时候，由于前面的吃是一个动词，那么很显然苹果作为名词的概率就会远大于动词的概率，因为动词后面接名词很常见，而动词后面接动词很少见。

为了解决这样类似的问题，更好的处理序列的信息，RNN就诞生了。

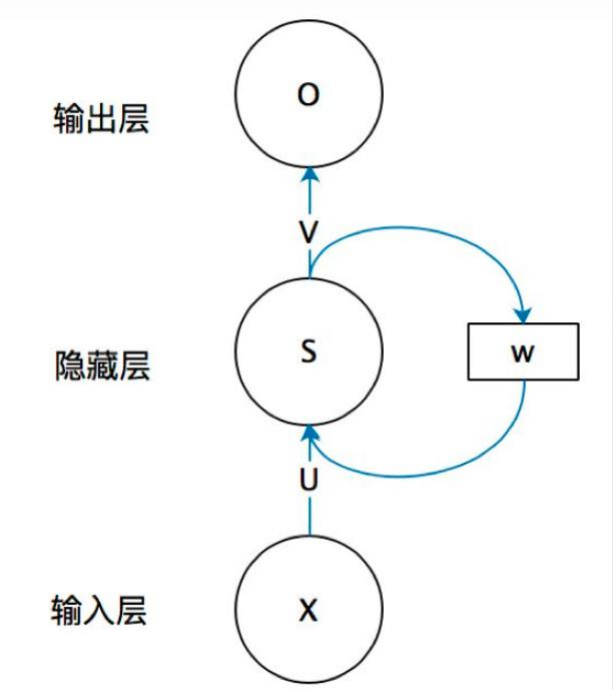
RNN的结构

- 首先看一个简单的循环神经网络，它由输入层、一个隐藏层和一个输出层组成：
- 它就变成了最普通的全连接神经网络。x是一个向量：输入层的值（这里没有画出来表示神经元节点的圆圈）；s是一个向量：隐藏层的值（这一层其实是多个节点）；U是输入层到隐藏层的权重矩阵，o是一个向量：输出层的值；V是隐藏层到输出层的权重矩阵。



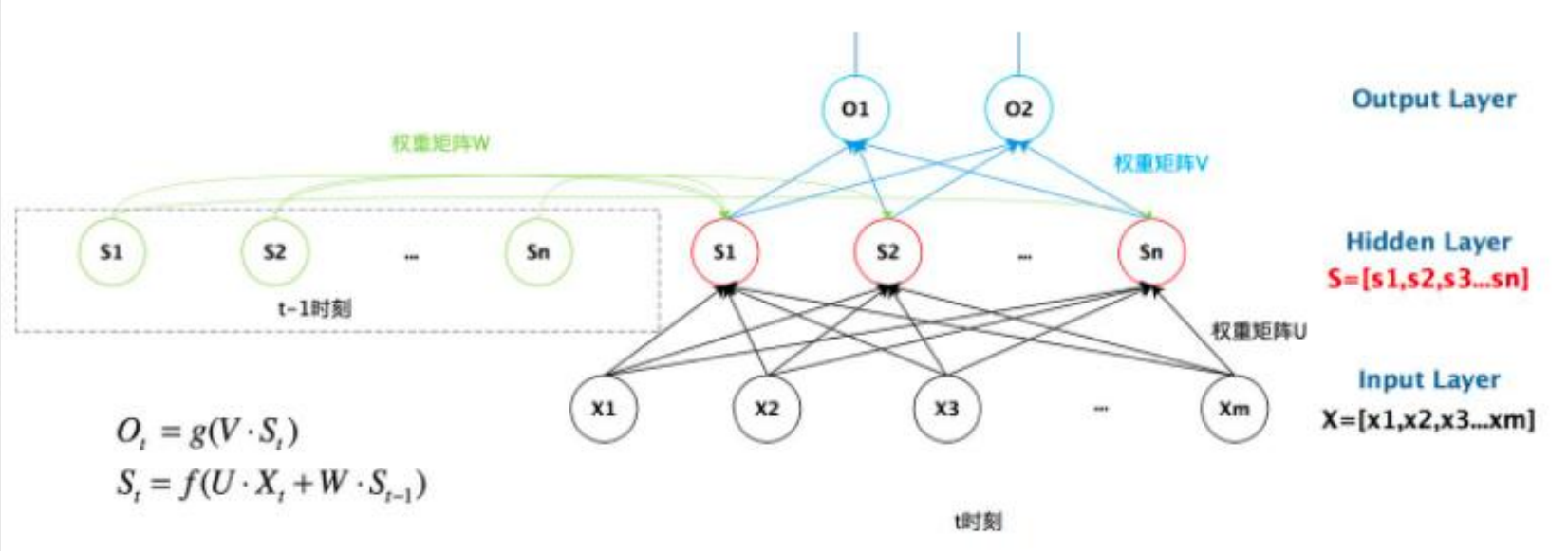
RNN的结构

- 现在我们来看看W是什么。循环神经网络的隐藏层的值s不仅仅取决于当前这次的输入x，还取决于上一次隐藏层的值s。权重矩阵 W就是隐藏层上一次的值作为这一次的输入的权重。



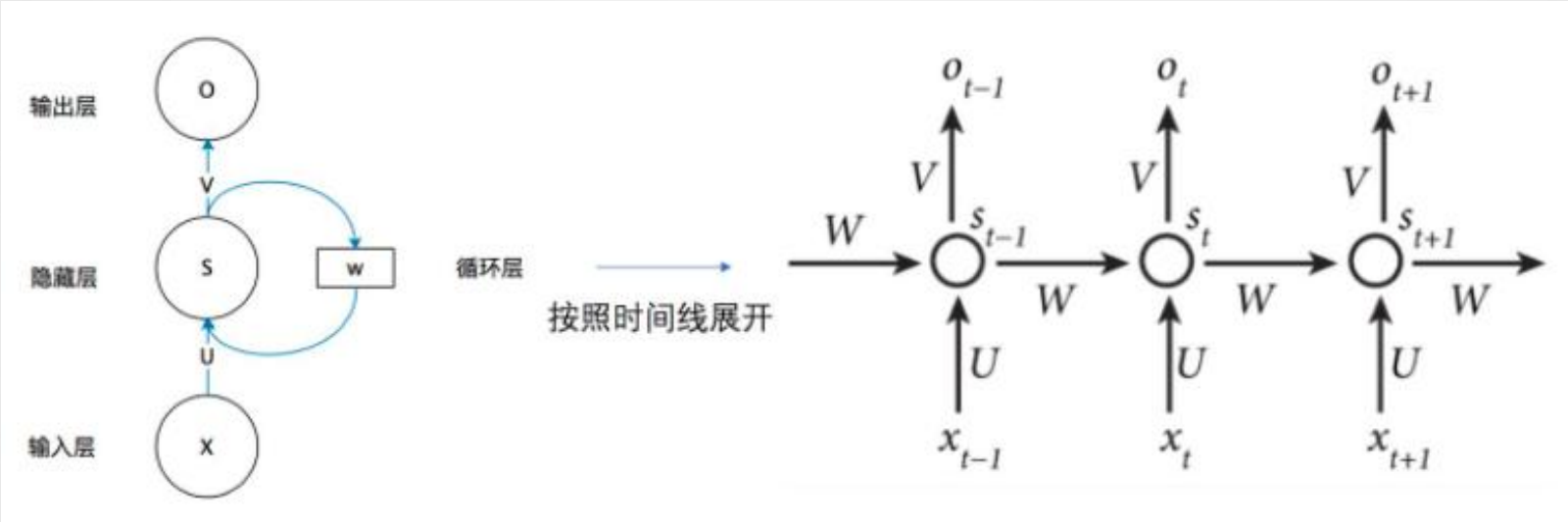
RNN的结构

- 我们给出这个抽象图对应的具体图：



RNN的结构

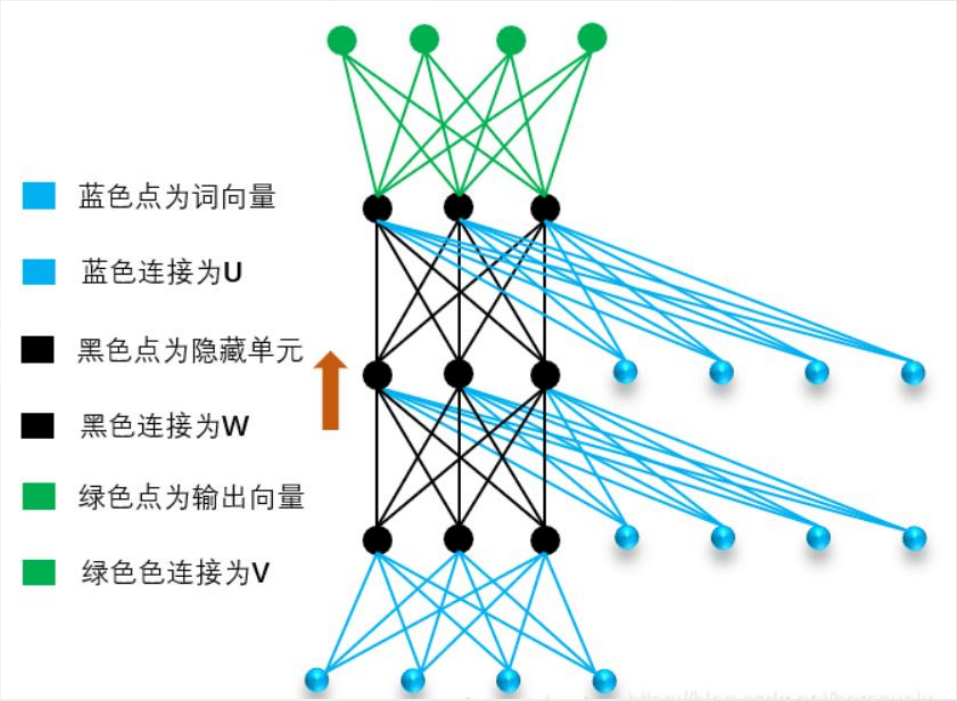
- 如果我们把上面的图展开，RNN也可以画成下面这个样子：



- 这个网络在t时刻接收输入x_t之后，隐藏层的值是s_t，输出值是o_t。关键一点：s_t的值不仅取决于x_t，还取决于 s_{t-1}

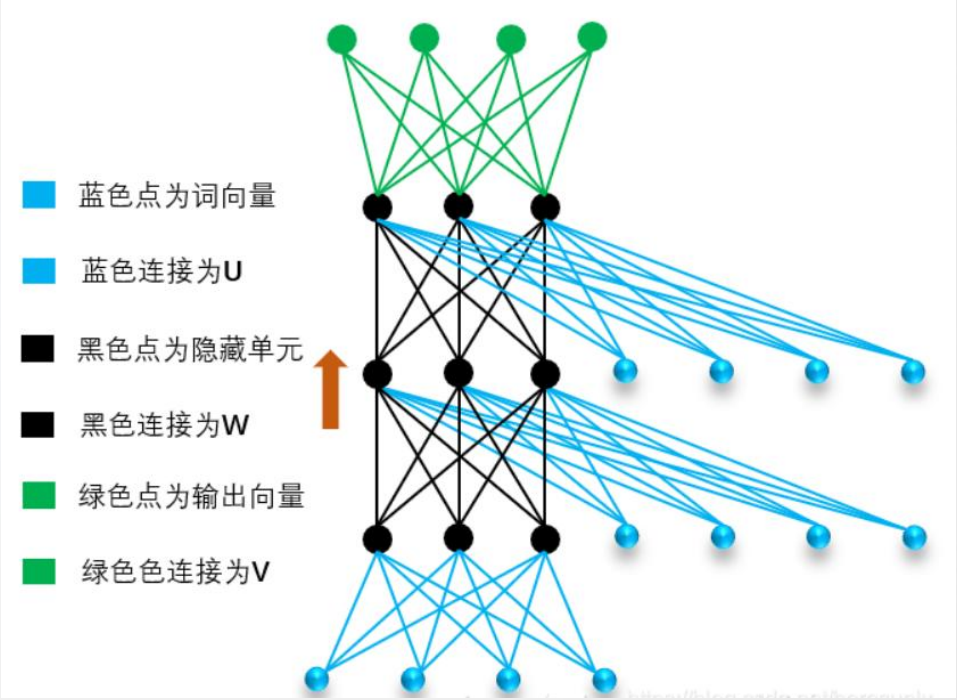
RNN的结构

- 下图为更直观详细的网络结构图：其中词向量**维度为4**，隐藏层为**3层**，隐藏层神经元个数为**3**，输出维度为**4**。RNN考虑了单词之间的顺序，对于每一步来说，隐藏层多了当前时刻的输入以及上一时刻的隐藏层。



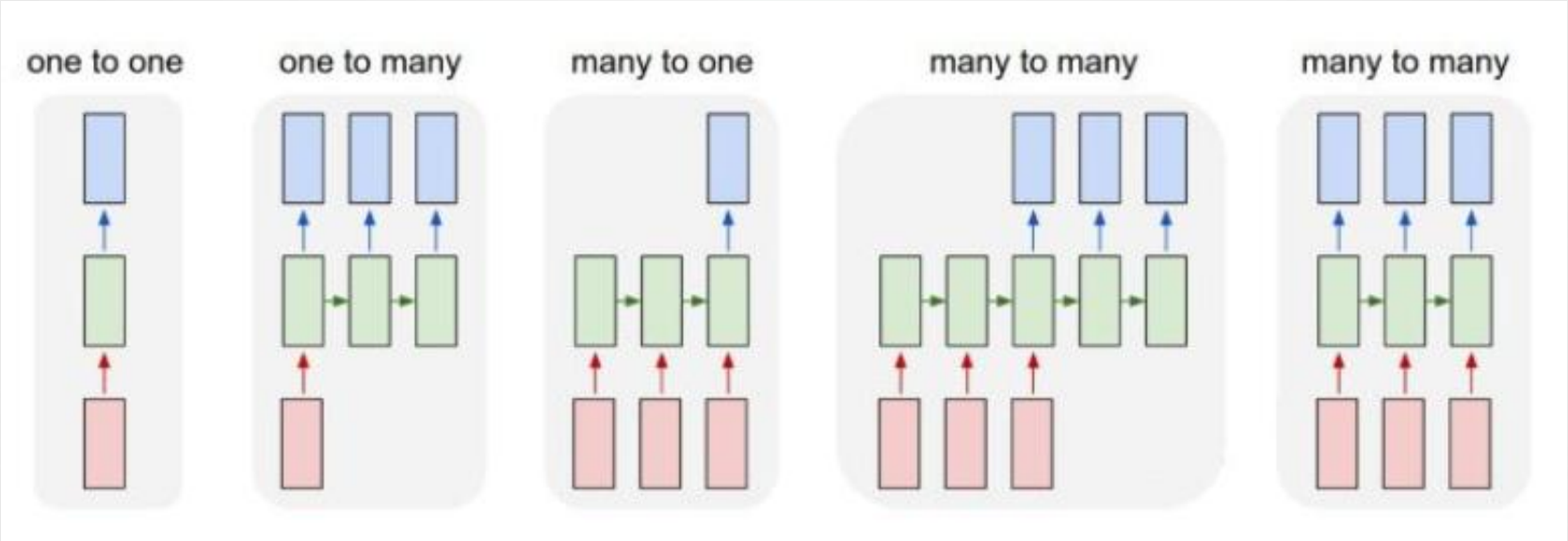
RNN的结构

- 可以看出RNN的巧妙之处：对于任意长度的输入，输出是一致的；能够表达序列之间的关系；参数共享易于训练。



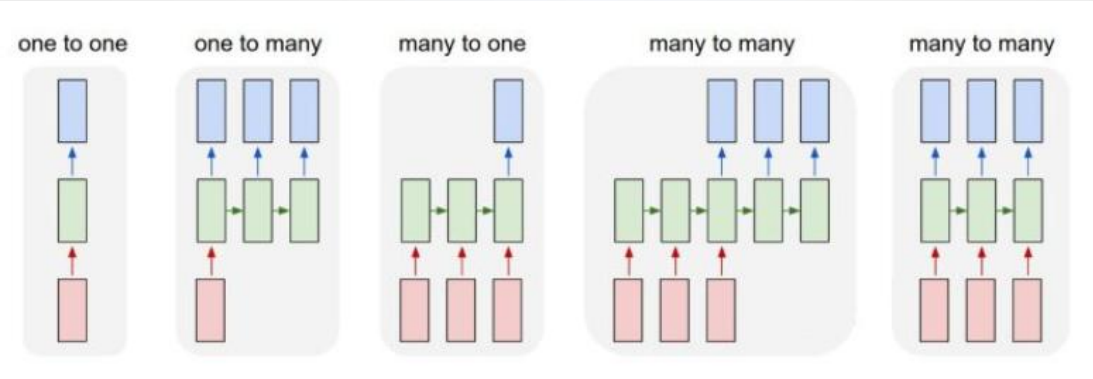
RNN的特点

- RNN的优点是使得序列处理成为可能：序列可以是输入，可以是输出，或者输入输出都是序列：



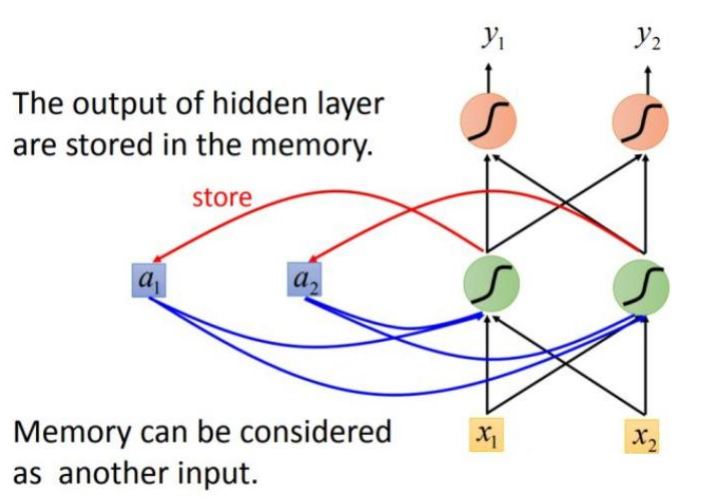
RNN的特点

- **第一个**是标准的神经网络，输入和输出都是固定的长度，譬如图像分类任务；**第二个**譬如输入一张图片，输出对图片的描述，图片本身作为一个向量，输出一个句子对它进行描述；**第三个**譬如对句子进行情感分析，输入是一个序列，输出只有一项；**第四个**譬如机器翻译，输入一个序列，输出一个序列；**第五个**譬如对视频进行每一帧的分类。



RNN计算举例

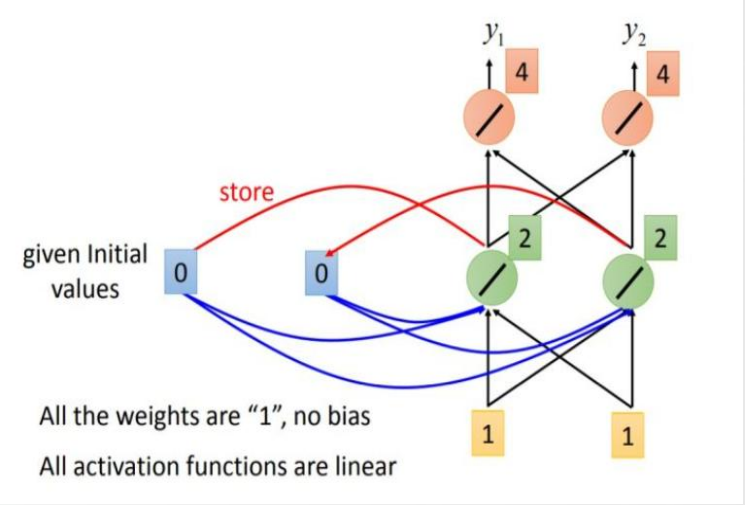
- 每个输入是**2*1的向量**；每个神经元产生一个**隐藏输入α**。在这里，假设所有的权重都是1，并且没有bias，所有的激活函数都是线性的。
- 那么在以下输入以及α初始值为0的情况下，计算每一步的结果。



- Input sequence: $\begin{bmatrix}1\\1\end{bmatrix}$ $\begin{bmatrix}1\\1\end{bmatrix}$ $\begin{bmatrix}2\\2\end{bmatrix}$ ……

RNN计算举例

- 上面是以输入[1,1]为例，算出来输出为[4,4]，同时更新α为[2,2]。
- 接下来，输入仍然为[1,1]，因为α为[2,2]，可以计算出隐藏层的结果是[6,6]，因此，输出层的结果是[12,12]，以及更新α为[6,6]。

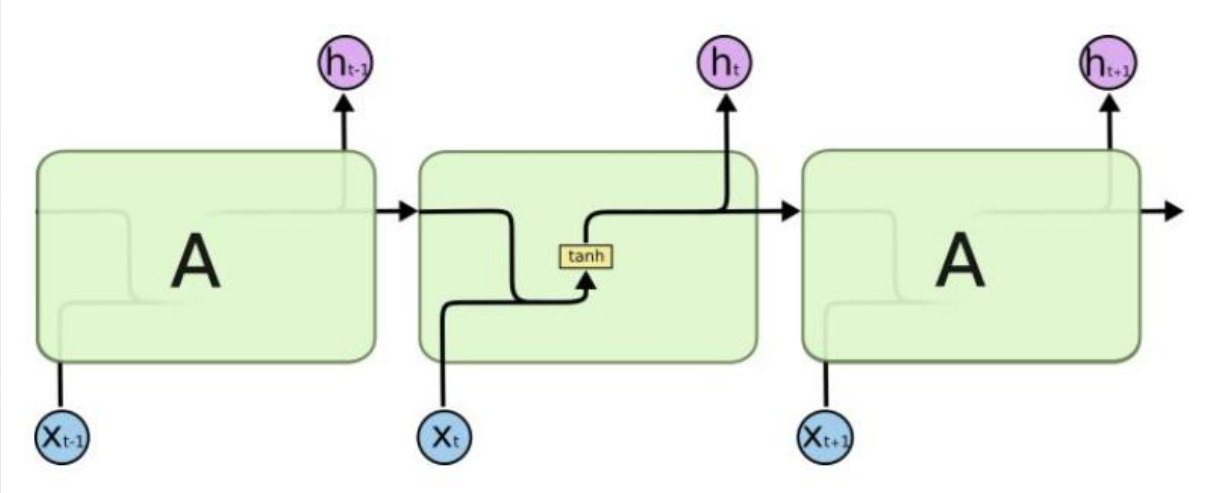


RNN计算举例

- 这里有一个很重要的RNN的特点，可以看到，连续两次输入的两个[1,1]，输出完全不一样。
- 可以猜想，如果更改了输入的顺序，那么输出的结果会完全不一样。所以，这就是RNN的优势所在。

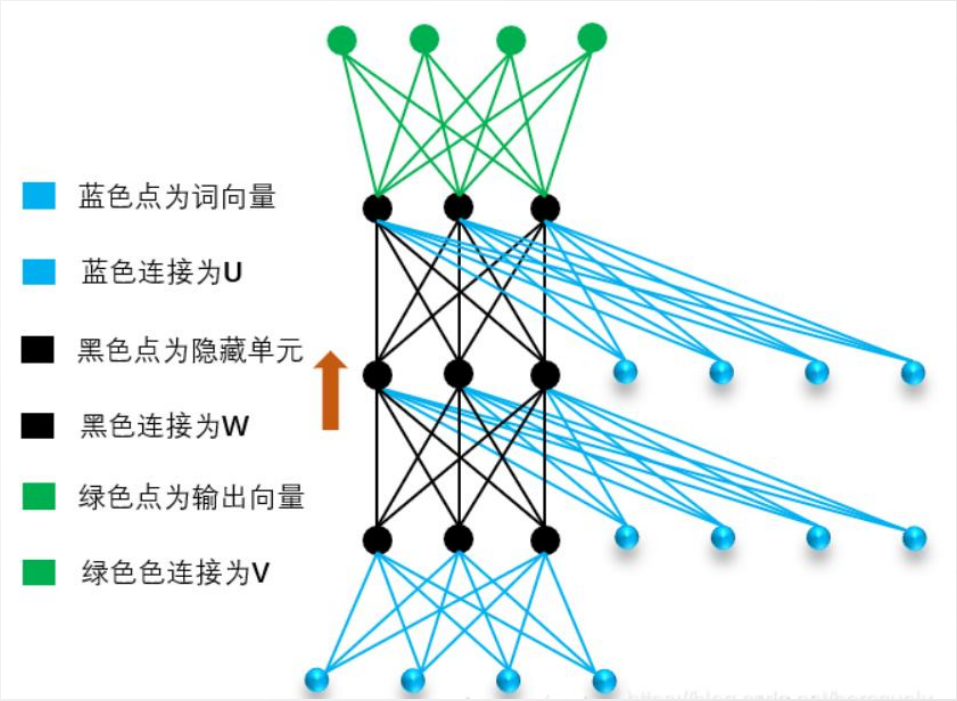
RNN如何记忆上下文?

- 可以从这张图(A表示神经网络)看出，之所以能够有记忆功能，主要是在 *t-1* 时刻产生的输出(可以理解为记忆)，作为输入参与了 *t* 时刻的计算。所以，如果不展开，那么看起来就像是在循环了，所以叫做循环神经网络。



RNN的问题

- 但是也发现了一个问题：RNN**对于长句子表达的不好**，第一个词和最后一个词的关系是通过好多次转换矩阵才产生的。



RNN到LSTM

- LSTM：Long Short-Term Memory，长短期记忆网络

有了RNN，为什么要提出LSTM呢?

想一下，人的日常行为流程是这样的：比如你在搭积木，那么每个时间点你的行为都会经历下面的子过程：
1、眼睛看到现在手里的积木。
2、回忆一下目前最高层的积木的场景。
3、结合1和2的信息来做出当前时刻积木插到哪里的决策。

第1步：手里的积木就是当前时刻的外部输入X；
第2步：就是调用历史时刻的信息/记忆；
第3步：就是融合X和历史记忆来推理出决策结果，即RNN的输出y(t)。

RNN到LSTM

- 有没有更加聪明的小伙伴惊奇的注意到第2步！！！
- 我们在回忆历史的时候，一般不是简单的回忆上一个积木的形状，而是去回忆一个更加模糊而宏观的场景。
- 在这个例子中，这个场景就是**最近几次行为所产生出的抽象记忆**——即“积木最高层的地形图”！

- 也就是说，人们在做很多时序任务的时候，并不是直接将上个时刻的输出y(t-1)直接连接进来，而是连接一个模糊而抽象的东西进来！
- 利用的是**一段历史记忆融合后的东西h，而不单单是上一时间点的输出**。

RNN到LSTM

- LSTM：Long Short-Term Memory，长短期记忆网络

有了RNN，为什么要提出LSTM呢?

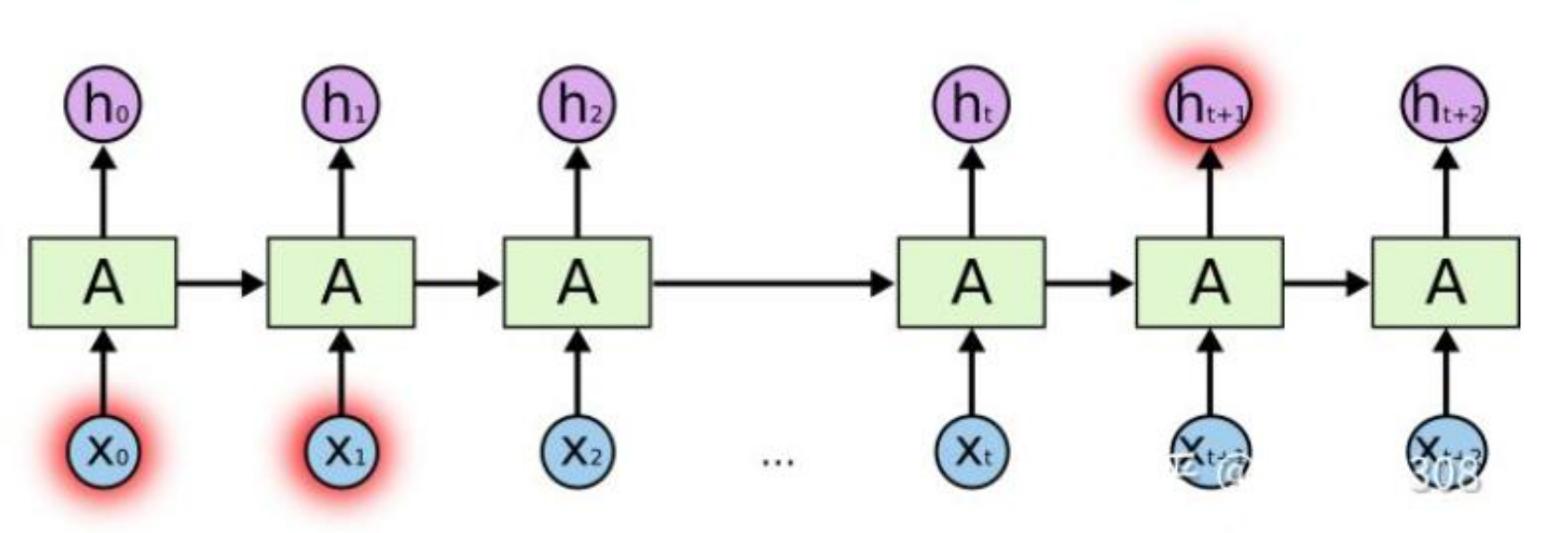
- 对于RNN而言，可以学习一些上下文知识，譬如“the clouds are in the sky,”很大概率上RNN能够预测出来，最后一个单词应该是sky，因为和sky相关的单词cloud距离sky**比较近**；

RNN到LSTM

- 像下面的句子：
  - *I grew up in France，… I speak fluent French。由France可知，后面的语言应该是French。*
- 再考虑一个例子：
  - *The cat, which ate a lot of fishes, is full。以及，*
  - *The cats, which ate a lot of fishes, are full。*
- 这两个句子之中的谓语，is 和 are，因为距离主语太远，在之前主要受附近的词影响的RNN网络中，就得不到有效的知识了。

RNN到LSTM

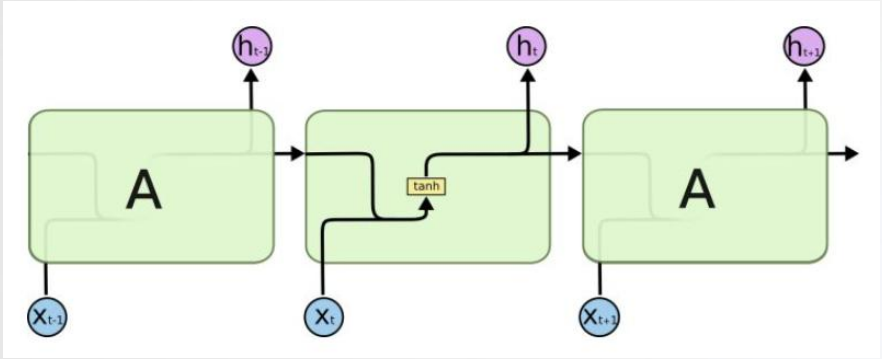
- 理解一下为什么隔得太远，RNN会失效



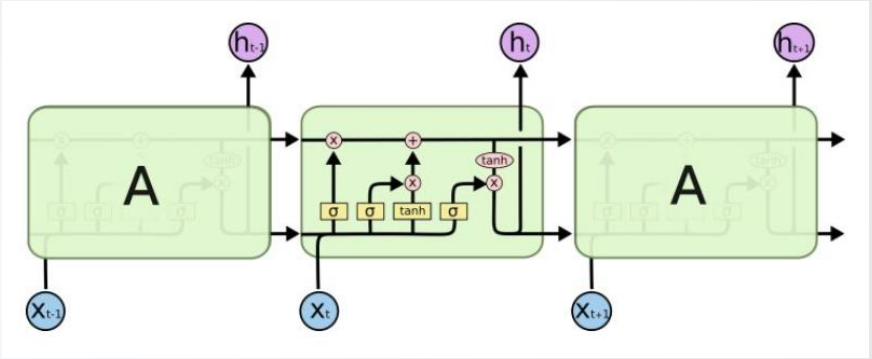
- 所以，LSTM提出的主要目标是**让网络具有较长远的记忆**。
  问题是，它是怎么样做到的呢?

LSTM如何具有长远记忆?

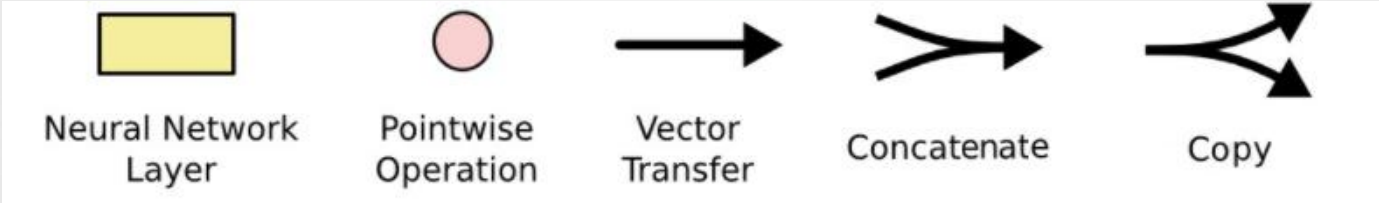
- LSTM和RNN的架构是一样的，但中间模块的结构不同。



RNN架构



LSTM的循环单元有四个网络结构，相互影响



LSTM如何具有长远记忆?

- 这里有三个**门**。为什么叫“门gate”呢?
- 因为比较像sigmoid的输出，在0~1之间，而sigmoid的输出都是进行点乘。
- 如果sigmoid的输出是0，意味着和它相乘的那个输入变成了0，**即这个输入不被通过**。
- 这里有三个门：forget gate，input gate和output gate。
  - input gate 用于控制有多少内容可以输入到cell中;
  - output gate用于控制有多少内容可以输出到新的cell中;
  - forget gate用于控制旧的cell中有多少内容可以保存。
- 至于门到底是打开还是关闭，就是网络自己学习到的。

LSTM如何具有长远记忆?

- LSTM的循环单元有四个网络结构，相互影响

1、遗忘门：应该丢弃什么信息

$$f_t = \sigma\left(W_f \cdot [h_{t-1},\ x_t] + b_f\right)$$

2、输入门：什么样的新信息存放在神经元中

$$i_t = \sigma\left(W_i \cdot [h_{t-1},\ x_t] + b_i\right)$$

$$\overline{C}_t = \tanh\left(W_C \cdot [h_{t-1},\ x_t] + b_C\right)$$

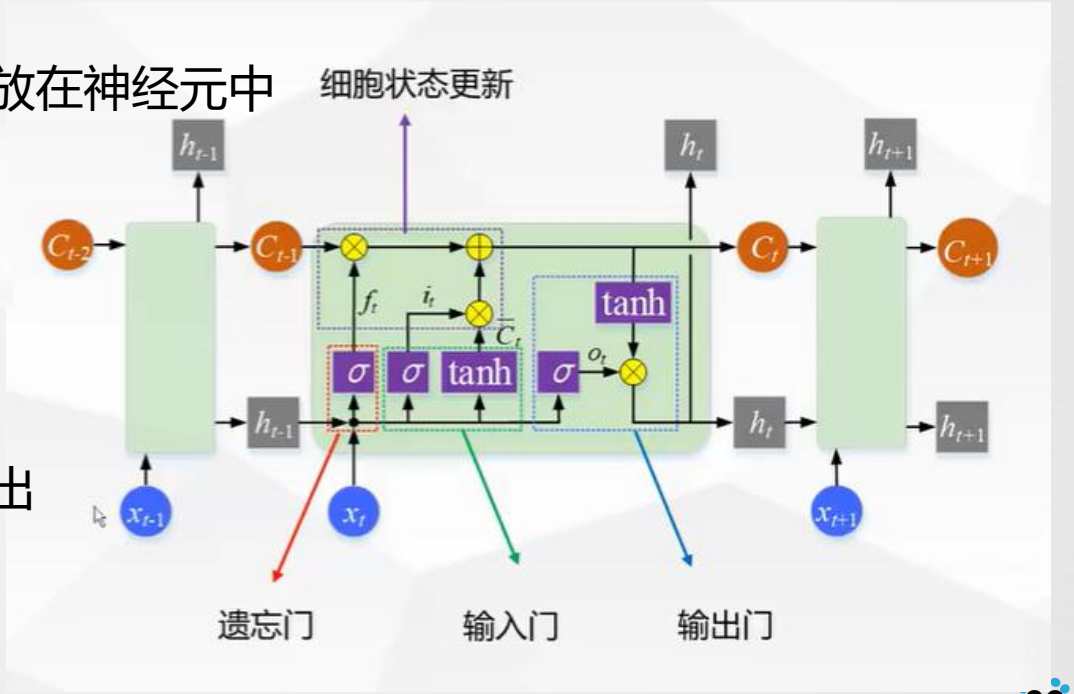
3、完成神经元状态更新

$$C_t = f_t C_{t-1} + i_t \overline{C}_t$$

4、输出门：决定什么信息被输出

$$o_t = \sigma\left(W_o \cdot [h_{t-1},\ x_t] + b_o\right)$$

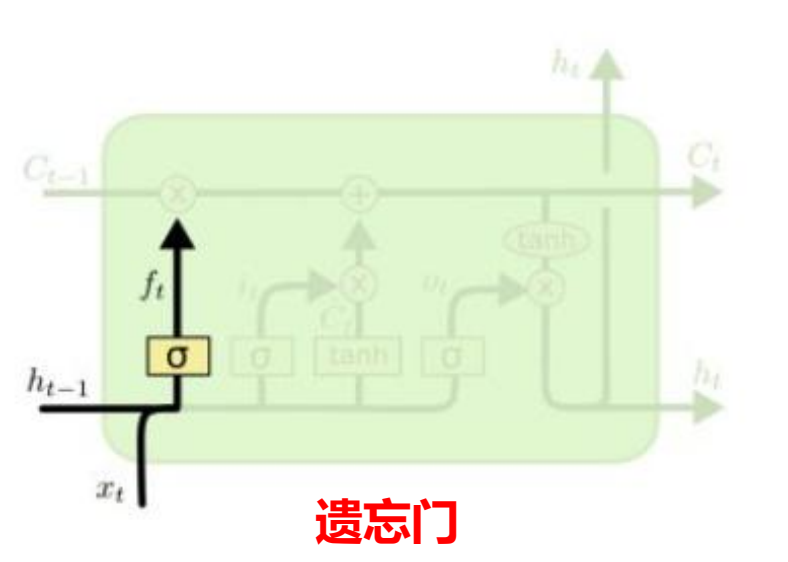
$$h_t = o_t \cdot \tanh\left(C_t\right)$$



LSTM如何具有长远记忆?

1、遗忘门：应该丢弃什么信息

- LSTM的第一步是决定需要从cell状态中扔掉什么样的信息。
- 由遗忘门的sigmoid层决定。
- 输入h_{t-1}和x_t，输出一个0和1之间的数。
- 1代表“完全保留这个值”，而0代表“完全扔掉这个值”。



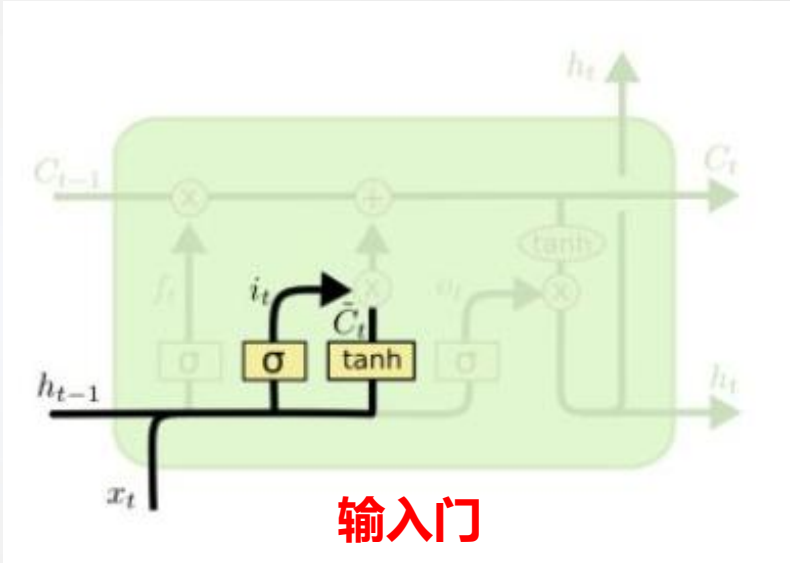
遗忘门

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

LSTM如何具有长远记忆?

2、输入门：什么样的新信息存放在神经元中

- 这个问题有两个部分：
- （1）一个sigmoid层调用“输入门”以决定哪些数据是需要更新的。
- （2）一个tanh层为新的候选值创建一个向量 $\bar{C}_t$ ，这些值能够加入state中。
- 下一步，我们要将这两部分合并以创建对state的更新。比如可以表示为想要把新的语言主题的信息加入到cell state中，以替代我们要遗忘的旧的记忆信息。



输入门

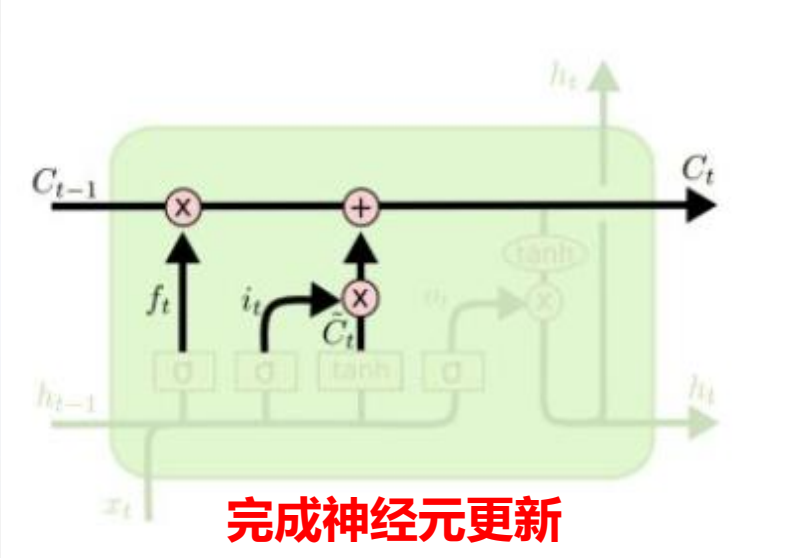
$$i_t = \sigma\left(W_i \cdot \left[h_{t-1},\ x_t\right] + b_i\right)$$

$$\bar{C}_t = \tanh\left(W_C \cdot \left[h_{t-1},\ x_t\right] + b_C\right)$$

LSTM如何具有长远记忆?

3、完成神经元状态更新

- 在决定需要遗忘和需要加入的记忆之后，就可以更新旧的cell state（C_{t-1}）到新的cell state（C_t）了。
- 在这一步，我们把旧的state C_{t-1}与f_t相乘，遗忘我们先前决定遗忘的东西，然后加上 $i_t\bar{C}_t$ ，这可以理解为新的记忆信息，
- 这里体现了新的输入对状态值的更新度是有限制的，我们可以把i_t当成一个权重。



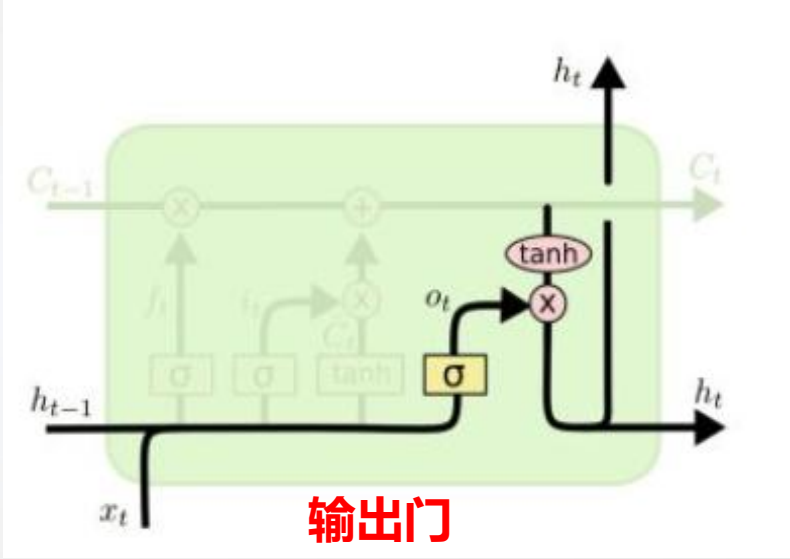
完成神经元更新

$$C_t = f_t C_{t-1} + i_t \bar{C}_t$$

LSTM如何具有长远记忆?

4、输出门：决定什么信息被输出

- 最后，我们需要决定要输出的东西。这个输出基于我们的cell state，但会是一个过滤后的值。
- 1）我们运行一个sigmoid层，这个也就是输出门，以决定cell state中的那个部分是我们将要输出的。
- 2）把cell state放进tanh（将数值压到-1和1之间），
- 3）最后将它与sigmoid门的输出相乘，这样就只输出了我们想要的部分了。



输出门

$$o_t = \sigma\left(W_o \cdot \left[h_{t-1},\ x_t\right] + b_o\right)$$

$$h_t = o_t \cdot \tanh\left(C_t\right)$$

LSTM如何具有长远记忆?

讲了这么多LSTM的原理，我们再回到最开始的几页介绍LSTM如何进行一词多义的词嵌入表示。

5 ELMO

ELMO的核心要点

阶段一

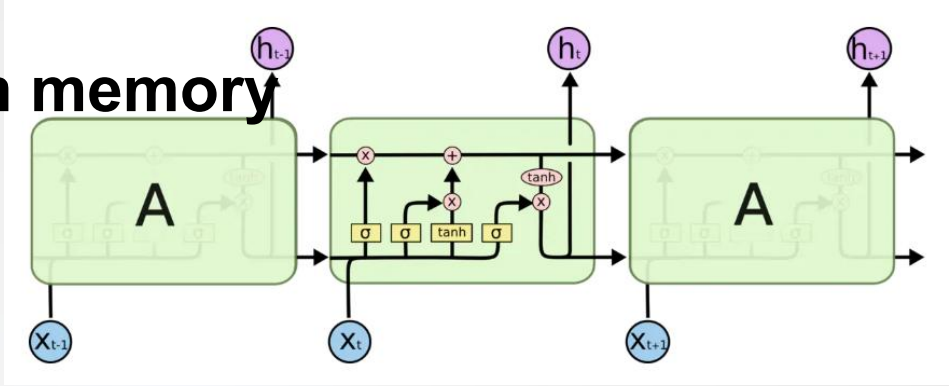
利用语言模型进行预训练：使用大规模的无标签语料库训练双向LSTM语言模型，ELMo（Embedding from Language Models）也是因此得名；

阶段二

将ELMo得到的特征向量送到下游任务中，得到任务相关的预测结果。

5 ELMO

LSTM：Long short-term memory



LSTM中重要的三个向量：
当前输入：X_t
当前输出：h_t
细胞状态：C_t

LSTM中重要的三个门：
(用于控制某个向量保留多少)
输入门：i_t
遗忘门：f_t
输出门：o_t

1、根据当前输入和上一时刻输出计算三个门：

$$i_t = \sigma(\mathbf{W_i} \cdot [\mathbf{h_{t-1}}, \mathbf{X_t}] + \mathbf{b_i})$$
$$f_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h_{t-1}}, \mathbf{X_t}] + \mathbf{b}_f)$$
$$o_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h_{t-1}}, \mathbf{X_t}] + \mathbf{b}_o)$$

2、计算当前细胞状态：

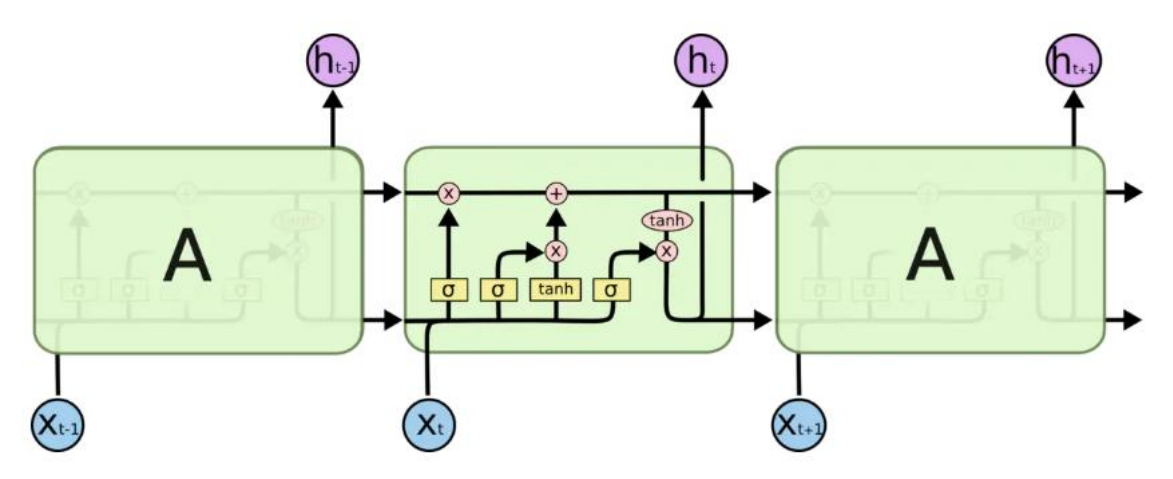
$$\widetilde{C_t} = \tanh(\mathbf{W}_C \cdot [\mathbf{h_{t-1}}, \mathbf{X_t}] + \mathbf{b}_C)$$
$$C_t = f_t \times C_{t-1} + i_t \times \widetilde{C_t}$$

3、计算输出值：

$$h_t = o_t * tanh(C_t)$$

5 ELMO

LSTM：Long short-term memory

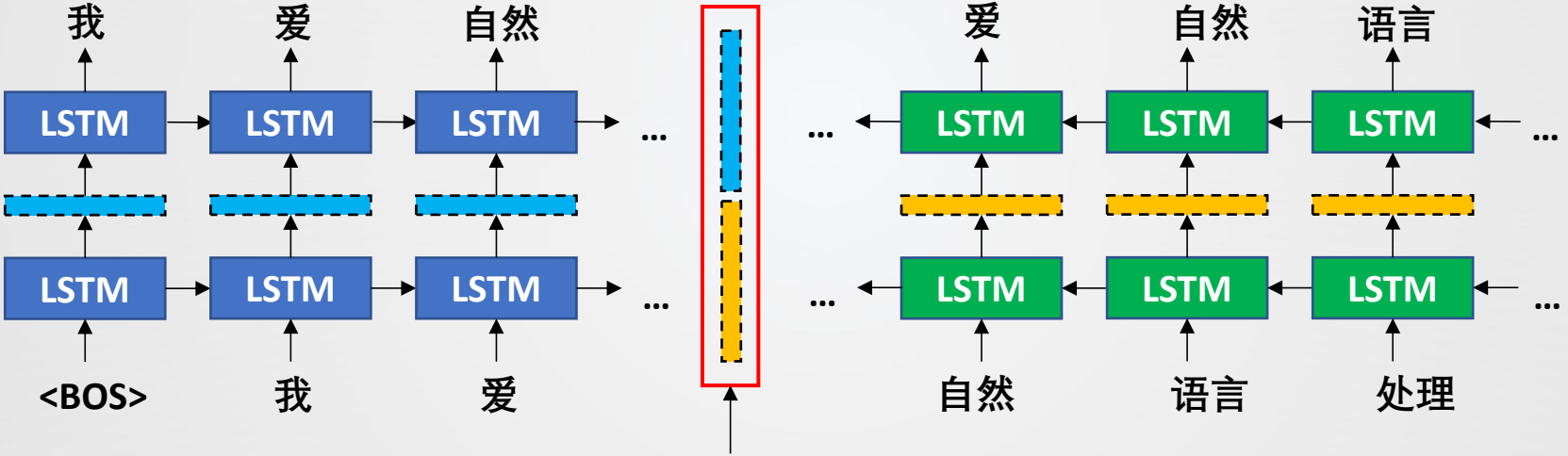


简单的理解：每个LSTM单元的输入是一个词(使用One-Hot或Word2Vec等方式向量化)，输出是根据当前输入和以往有用的信息所确定的一个向量。可以看做把词向量进一步编码成了对当前任务更有用的向量。

5 ELMO

ELMo在预训练时做的任务就是通过上一个词预测下一个词

例子：我 爱 自然 语言 处理。

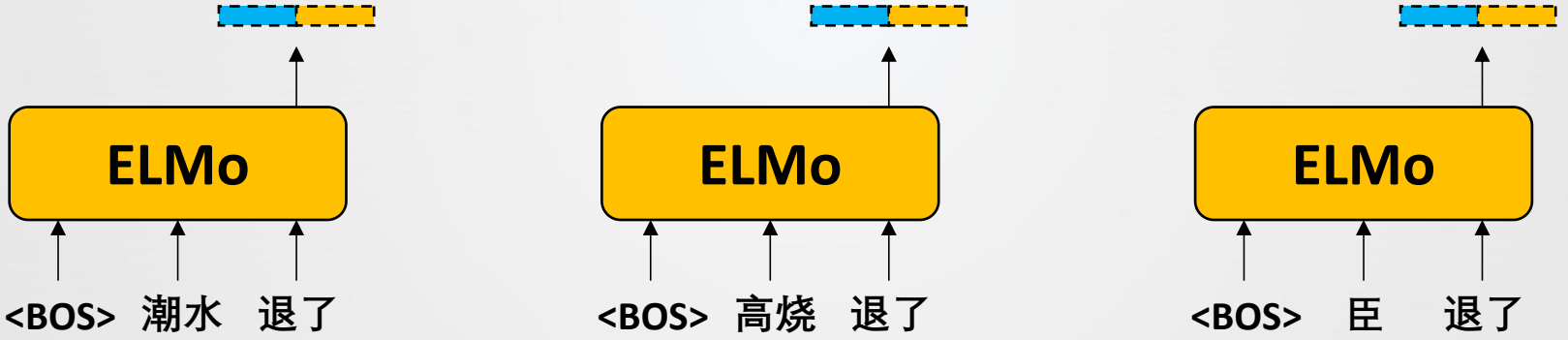


正向和反向的拼接则是我们要得到目标向量

5 ELMO

为什么ELMo能够做到“一词多义”？

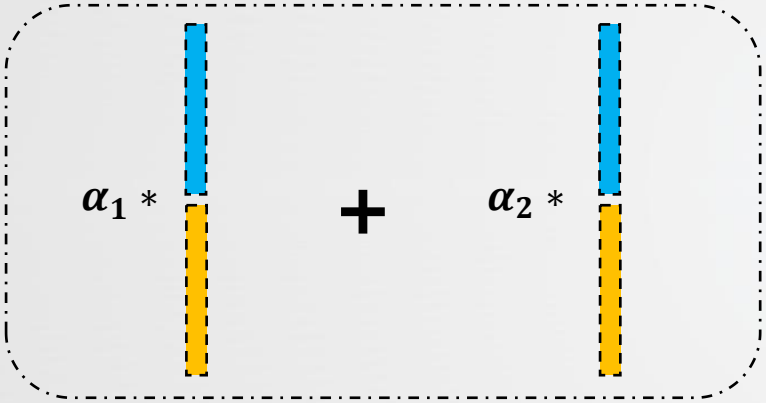
- 因为ELMo中的向量不仅仅是根据词的样子得到的，还根据该词所处的上下文。
- 举例：下面三个“退了”虽然都是同一个词，但是因为所处的上下文不同，表示的向量也不完全相同。



5 ELMO

在ELMo中有很多层的LSTM，那要怎么选择呢?

最终的词向量:



第一层LSTM输出　第二层LSTM输出



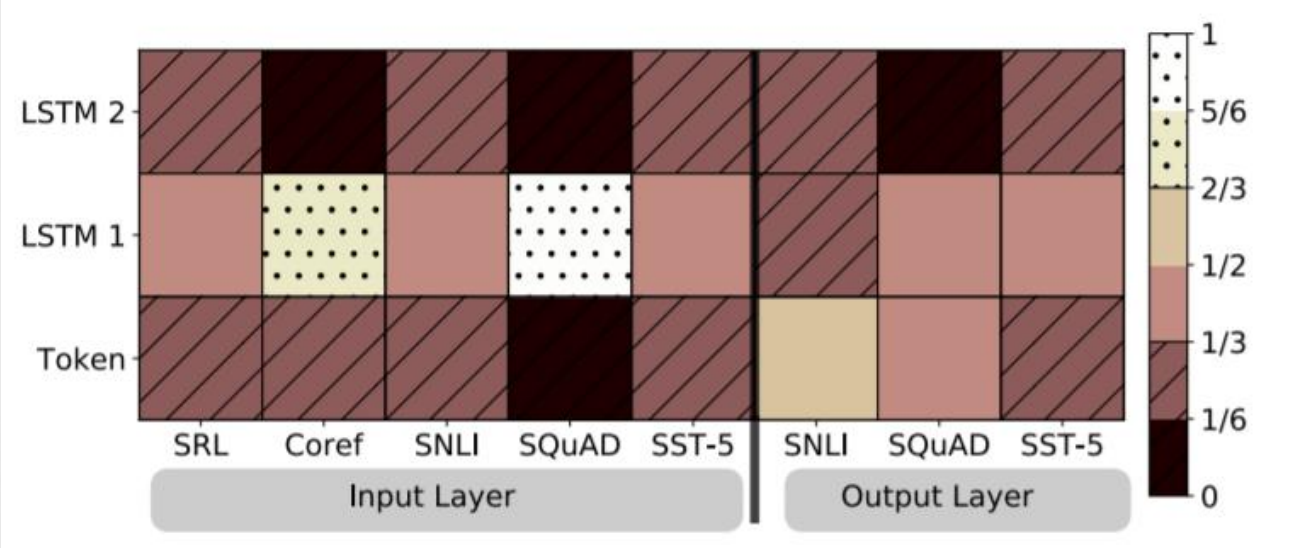
- α_1和α_2是怎么来的呢？答案是**训练得来的**！
- 注意到：通过上一个词预测下一个词的任务称**预训练(pre-train)**而不是训练，因为这只是一个预先准备的任务，并不是我们真正的目标。
- 在完成预训练之后还需要根据我们的目标进行**微调(fine tune)**。比如：QA、分类等，还需要再针对我们的目标任务，再进行一次针对性的训练，而α_1和α_2 就是在这其中训练得到的。

5 ELMO

预训练vs微调：

预训练： 训练机器识字。

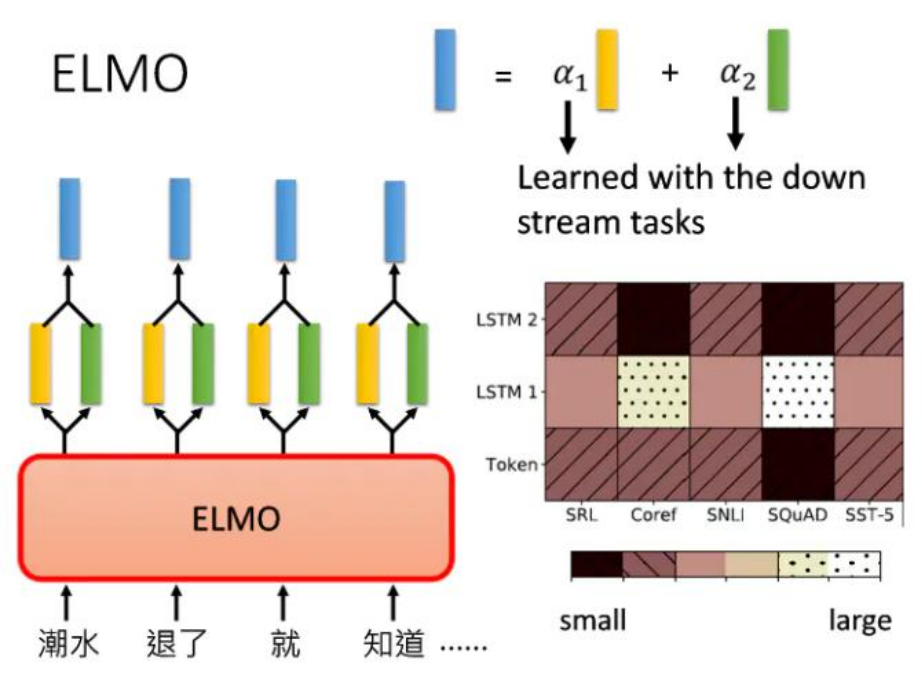
微　调： 训练机器写作文、训练机器判案、训练机器进行日常问答等。



ELMo针对不同数据集给出了不同α_1和α_2权重，不同任务对于不同层次的词向量关注的重点也不同

5 ELMO

预训练vs微调：



- 在上下文embedding的训练模型中，每个词输入进去都会有一个embedding输出。
- 但在ELMo中，每个词输入进去，会得到不止一个embedding，因为每层的RNN都会给到一个embedding，ELMo将它们统统加起来一起使用。
- 以图中为例，这里假设ELMo有两层LSTM，这里将α1(黄色，第一层得到的embedding)和α2(绿色，第二层得到embedding)加起来得到蓝色的embedding，并做为接下来要进行不同任务的输入。

5 ELMO

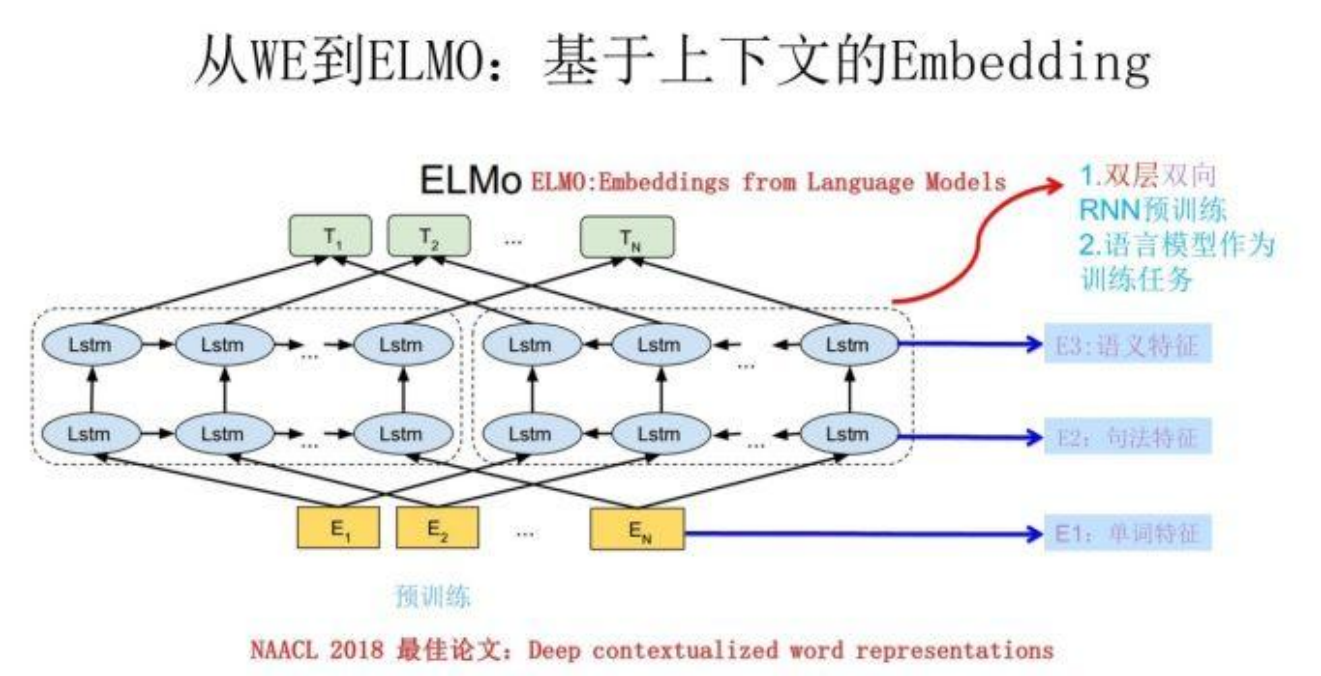
预训练vs微调：

- 这里存在一些问题：α1和α2是学习得到的，而且它是根据当前要进行的任务(如QA，POS of tagging)，然后根据接下来要进行的任务一起被学习出来。所以就导致不同任务导向下的α1和α2也不一样。

- ELMo的论文中提到，在不同任务下(SRL,Coref,SNLI,SQuAD,SST-5)，蓝色的上下文embedding在经过token(这里为没有经过上下文的embedding)，LSTM1，LSTM2后，它在不同阶段需要的weight也不一样。

5 ELMO接下游任务举例

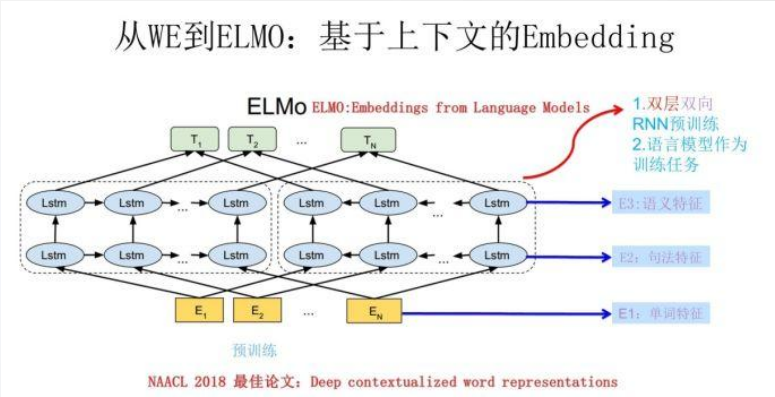
ELMO第一阶段：



5 ELMO接下游任务举例

ELMO第一阶段：

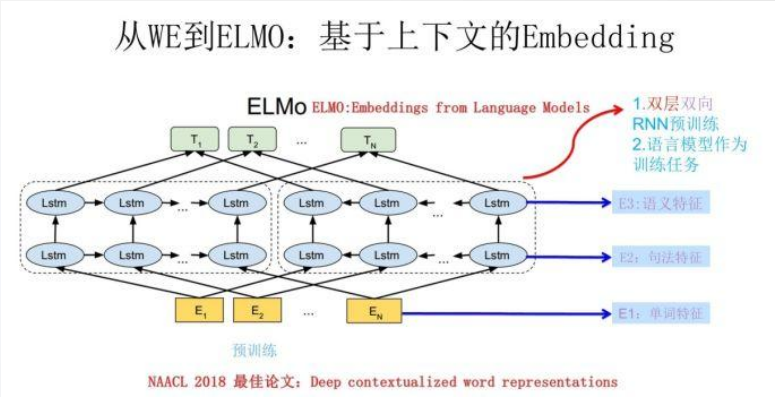
- 第一个阶段是利用语言模型进行预训练，右图展示的是其预训练过程，它的网络结构采用了双层双向LSTM，目前语言模型训练的任务目标是根据单词的上下文去正确预测单词，之前的单词序列Context-before称为上文，之后的单词序列Context-after称为下文。



5 ELMO接下游任务举例

ELMO第一阶段：

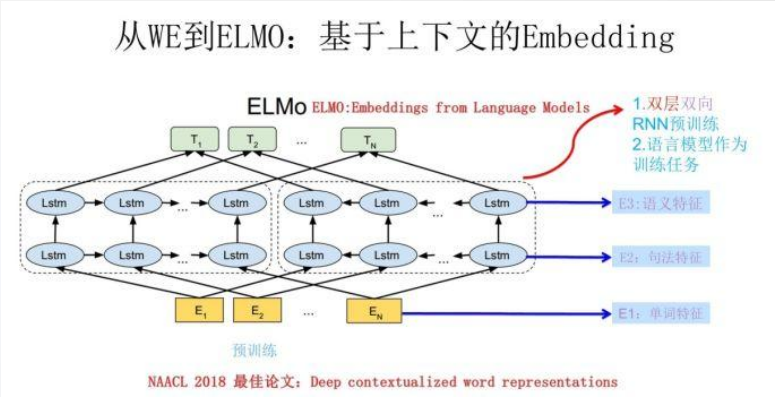
- 图中左端的前向双层LSTM代表正方向编码器，输入的是从左到右顺序的除了预测单词外的上文Context-before；
- 右端的逆向双层LSTM代表反方向编码器，输入的是从右到左的逆序的句子下文Context-after；
- 每个编码器的深度都是两层LSTM叠加。



5 ELMO接下游任务举例

ELMO第一阶段：

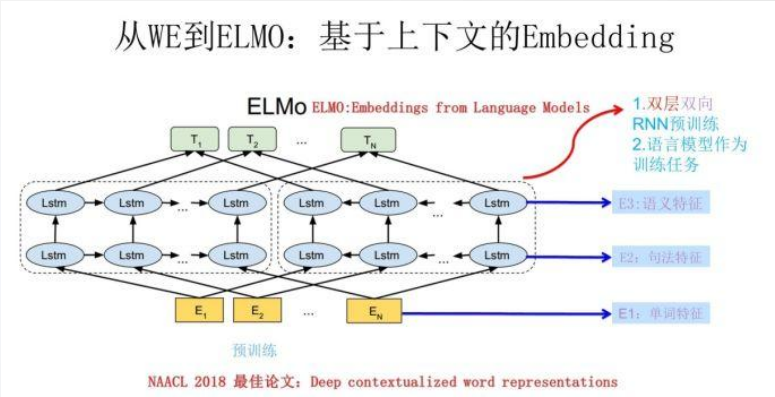
- 使用这个网络结构利用大量语料预先训练好这个网络，之后输入一个新句子，句子中每个单词都能得到三个Embedding：
  - 最底层是单词的Word Embedding，
  - 往上一层对应单词位置的Embedding，这层编码单词的**句法信息**更多；
  - 再往上一层LSTM中对应单词位置的Embedding，这层编码单词的**语义信息**更多。



5 ELMO接下游任务举例

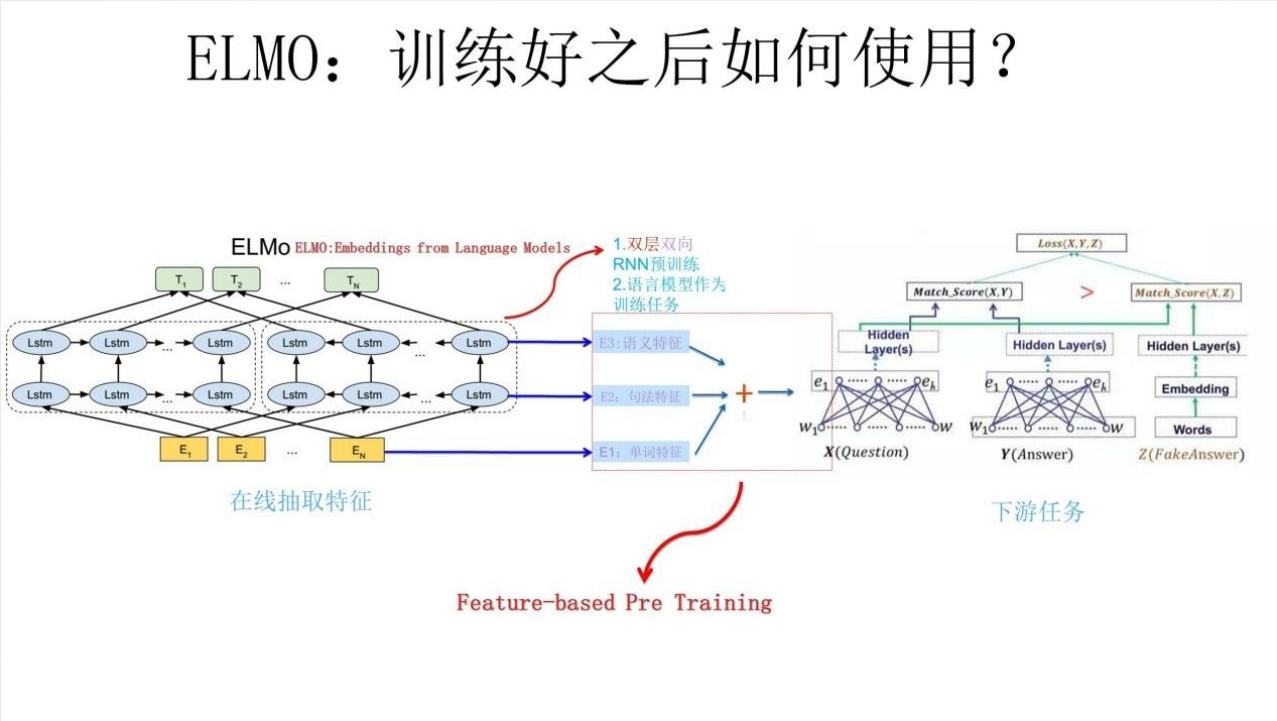
ELMO第一阶段：

- 也就是说，ELMO的预训练过程不仅仅学会单词的Word Embedding，还学会了一个双层双向的LSTM网络结构，而这两者在ELMO第二阶段都会用到。



5 ELMO接下游任务举例

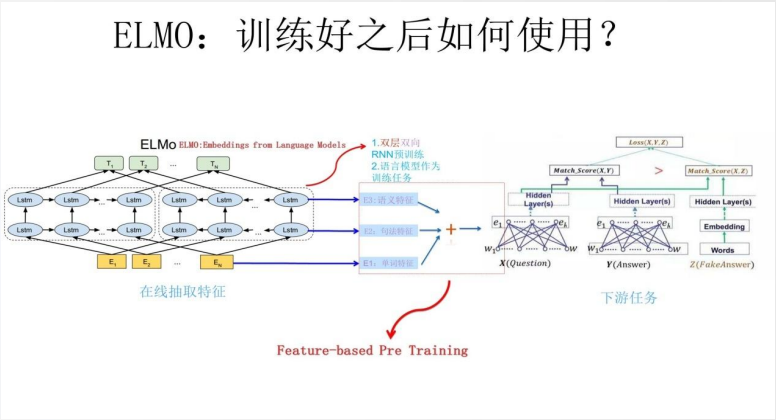
ELMO第二阶段：



5 ELMO接下游任务举例

ELMO第二阶段：

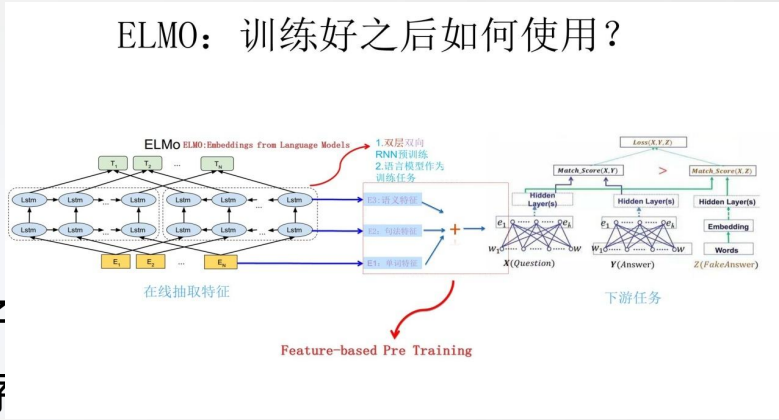
- 右图展示了下游任务的使用过程，比如下游任务是QA问题，此时对于问句X，先将句子X作为预训练好的ELMO网络的输入，这样句子X中每个单词在ELMO网络中都能获得对应的三个Embedding，之后给予这三个Embedding中不同的权重（这个权重可以通过学习得来），根据各自权重累加求和，将三个Embedding**整合成一个。**



5 ELMO接下游任务举例

ELMO第二阶段：

- 然后将整合后的这个Embedding作为X句在自己任务的那个网络结构中对应单词的输入，以此作为补充的新特征给下游任务使用。
- 对于上图所示下游任务QA中的回答句子Y来说也是如此处理。因为ELMO给下游提供的是每个单词的特征形式，所以这一类预训练的方法被称为“Feature-based Pre-Training”。



5 ELMO—词多义

思考

为何这么做能够达到区分多义词的效果?



5 ELMO—词多义

效果

ELMO：效果如何？

Task	Previous SOTA		Our baseline	ELMo + baseline	Increase (absolute/ relative)
SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

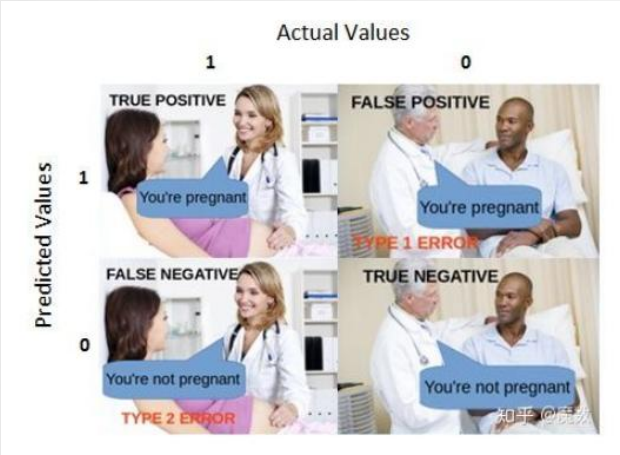
6个NLP任务：5%到25%的提高！

- 效果见上图，6个NLP任务中性能都有不同幅度的提升，最高的提升达到25%左右，而且这6个任务的覆盖范围比较广，包含句子语义关系判断、分类任务、阅读理解等多个领域，这说明其适用范围是非常广的，这相比之前的模型提升效果明显。

5 ELMO的优缺点

- 优点：
  - ✓ 效果好，在大部分任务较传统模型都有一定提升
  - ✓ 解决了以往词向量不能很好表示**“一词多义”**的问题
- 缺点：
  - × 速度较慢，需要通过多层双向LSTM进行训练
  - × ELMO网络架构中LSTM抽取特征能力远弱于Transformer（在GPT和Bert中使用）
  - × 拼接方式采用双向融合，特征融合能力偏弱

6 混淆矩阵



样本		预测值	
		1	0
实际值	1	TP(真正)	FN(假负)
	0	FP(假正)	TN(真负)

6 混淆矩阵

- TP表示预测值为正（positive），实际值也为正（positive）；
- FP表示预测值为正(positive)，实际值为反（negative）；
- FN表示预测值为反（negative），实际值为正（positive）；
- TN表示预测值为反（negative），实际值也为反（negative）。

样本		预测值	
		1	0
实际值	1	TP(真正)	FN(假负)
	0	FP(假正)	TN(真负)

6 混淆矩阵

- 对于预测性分类模型，我们肯定是希望预测结果越准越好。
- 那么，对应到混淆矩阵中，就是希望TP与TN对应位置的数值越大越好，而FP与FN对应位置的数值越小越好。
- 但在模型结果预测中，单看这些对应位置的数值大小往往不够直观，
- 所以我们通常会通过以下几个指标来判定模型结果的好坏。

样本		预测值	
		1	0
实际值	1	TP(真正)	FN(假负)
	0	FP(假正)	TN(真负)

6 混淆矩阵

- 1.准确率(Accuracy) = (TP + TN) / (TP + FP + FN + TN)
  （在整个观察结果中，预测正确的占比）
- 2.精确率(Precision) = TP / (TP + FP)（在所有预测为正例的结果中，预测正确的占比）
- 3.召回率(Recall) = TP / (TP + FN)（在所有实际值为正例的结果中，预测正确的占比）
- 4.F1_score = 2*P*R/(P + R)(P代表精确率，R代表召回率)

样本		预测值	
		1	0
实际值	1	TP(真正)	FN(假负)
	0	FP(假正)	TN(真负)

注意：1、以上几个指标范围在0-1之间，数值越大表示相应结果越好；2、精确率是针对预测结果而言的，召回率是针对实际结果而言的；